



A Certified Multi-prover Verification Condition Generator

Paolo Herms, Claude Marché, Benjamin Monate

► To cite this version:

Paolo Herms, Claude Marché, Benjamin Monate. A Certified Multi-prover Verification Condition Generator. [Research Report] RR-7793, INRIA. 2011, pp.22. <hal-00639977>

HAL Id: hal-00639977

<https://inria.hal.science/hal-00639977v1>

Submitted on 10 Nov 2011

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization



A Certified Multi-prover Verification Condition Generator

Paolo Herms , Claude Marché, Benjamin Monate

**RESEARCH
REPORT**

N° 7793

November 2011

Project-Teams PROVAL



A Certified Multi-prover Verification Condition Generator

Paolo Herms^{* † ‡}, Claude Marché^{† ‡}, Benjamin Monate^{*}

Project-Teams PROVAL

Research Report n° 7793 — November 2011 — 19 pages

Abstract: Deduction-based software verification tools have reached a maturity allowing them to be used in industrial context where a very high level of assurance is required. This raises the question of the level of confidence we can grant to the tools themselves. We present a certified implementation of a verification condition generator. An originality is its genericity with respect to the logical context, which allows us to produce proof obligations for a large class of theorem provers. This implementation is conducted within the Coq proof assistant, and is crafted so that it can be extracted into a standalone executable, independent of Coq, which is another originality.

Key-words: Deductive Verification, Weakest Precondition Calculus, Coq Proof Assistant

This work is partly supported by the *U3CAT* (ANR-08-SEGI-021, <http://frama-c.com/u3cat/>) project of the French national research organization (ANR)

^{*} CEA, LIST, Lab. de Sûreté du Logiciel, Gif-sur-Yvette F-91191

[†] INRIA Saclay - Île-de-France, F-91893

[‡] Lab. de Recherche en Informatique, Univ Paris-Sud, CNRS, Orsay, F-91405

**RESEARCH CENTRE
SACLAY – ÎLE-DE-FRANCE**

Parc Orsay Université
4 rue Jacques Monod
91893 Orsay Cedex

Un générateur d'obligations de preuve certifié et multi-prouveurs

Résumé : Les outils de vérification de programme basés sur la preuve ont atteint un nouveau de maturité permettant leur utilisation dans un contexte industriel où un haut niveau de confiance est requis. Cela soulève la question du niveau de confiance que l'on peut mettre dans les outils eux-mêmes. Nous décrivons une implémentation certifiée d'un générateur d'obligations de preuve. Une originalité est sa genericité vis-à-vis du contexte logique, permettant de générer des obligations pour une grande famille de prouveurs. Cette implémentation est réalisée avec l'assistant à la preuve **Coq**, et est conçue dans l'optique d'en extraire un exécutable indépendant de Coq, garantit correct, ce qui est une autre originalité.

Mots-clés : Vérification déductive, calcul de plus faible pré-condition, assistant de preuve Coq

Contents

1	Introduction	5
2	Logical Contexts	6
2.1	Logical Signatures	6
2.2	Dependently Typed <i>de Bruijn</i> Indices	7
2.3	Terms and Propositions	8
2.4	Logical Contexts, Semantics	8
3	The Core Programming Language	9
3.1	Informal Description	9
3.2	Formal Syntax of Expressions	10
3.3	Operational Semantics	11
4	Weakest Precondition Calculus	12
4.1	Effect Inference	12
4.2	Definition of the WP-calculus	13
4.3	Soundness Results	14
5	Extraction of a Certified Verification Tool	14
5.1	Concrete WP computation	15
5.2	Producing Concrete Syntax with Explicit Binders	16
5.3	Extraction and Experimentation	16
6	Conclusions, Related Works and Perspectives	17

List of Figures

1	Logical context for sorting	6
2	Inductive definitions of terms and propositions	8
3	Denotational semantics of terms and propositions	9
4	Syntax of concrete expressions	10
5	Selection sort in our core language	10
6	Inductive definition of expressions	10
7	Operational semantics of terminating expressions	11
8	Operational semantics of non-terminating expressions	12
9	Recursive definition of the WP-calculus	13

1 Introduction

Among the various classes of approaches to static verification of programs, the so-called *deductive verification* approach amounts to verifying that a program satisfies a given behavioral specification by means of theorem proving. Typically, given a program and a formal specification, a verification condition generator produces a set of logical formulas, that must be shown to be valid by some theorem prover. Deductive verification tools have nowadays reached a maturity allowing them to be used in industrial context where a very high level of assurance is required [24]. This raises the question of the level of confidence we can grant to the tools themselves. This is the question we address in this paper.

One can distinguish two main kinds of deductive verification approaches. The first kind is characterized by the use of a deep embedding of the input programming language in a general purpose proof assistant. One of the earlier work of this kind is done in the *SunRise* system in 1995 [16] where a simple imperative language is defined in HOL, with a formal operational semantics. A set of Hoare-style deduction rules are then shown valid. A *SunRise* program can then be specified using HOL assertions, and proved in the HOL environment.

The second kind of approaches provide standalone verification condition generators automatically producing verification conditions, usually by means of variants of Dijkstra's weakest precondition calculus. This is the case of ESC/Java [10], B [1] ; the *Why* platform [14] and its Java [21] and C [13] front-ends ; and *Spec#* [3] and *VCC* [11] which are front-ends to *Boogie* [2]. Being independent of any underlying proof assistant, these tools analyze programs where formal specifications are given in ad-hoc annotation language such as *JML* [7] and *ACSL* [4]. However, up to now these standalone tools have never been formally proved to be sound.

Our goal is to combine the best of both approaches: a guaranteed sound VC generator, able to produce VCs for multiple provers. We implement and prove sound, in the *Coq* proof assistant [5], a VC generator inspired by the former *Why* tool. To make it usable with arbitrary theorem provers as back-ends, we make it generic with respect to a *logical context*, containing arbitrary abstract data types and axiomatizations. Such a generic aspect is suitable to formalize memory models needed to design front-ends for mainstream programming language, as it is done for C by *VCC* above *Boogie* or *Frama-C/Jessie* above *Why*. The input programs of our VC generator are imperative programs written in a core language which operates on mutable variables whose values are data types of the logical context. The output logic formulas are built upon the same logical context. This certified *Coq* implementation is crafted so it can be extracted into a standalone executable.

Section 2 formalizes our notion of generic logical contexts. Section 3 formalizes our core language, and defines its operational semantics. Section 4 defines the weakest precondition computation *WP* and proves its *soundness*. Theorem 13 states that if for each function of a program, its precondition implies the *WP* of its post-condition, then all its annotations are satisfied. Section 5 aims at the extraction of a standalone executable. We introduce a variant *wp* of the calculus which produces concrete formulas instead of *Coq* ones. Theorem 16 states that *wp* obligations imply the *WP* obligations. The main result is then Theorem 17 which states the soundness of the complete VC generation process. We conclude in Section 6 by comparing with related works and discussing perspectives. The sources of the underlying *Coq* development are available at <http://www.lri.fr/~herms>.


```

type array
logic select : array -> int -> int
logic store :
  array -> int -> int -> array
axiom select_eq: forall a,i,x.
  select (store a i x) i = x
axiom select_neq : forall a,i,j,x.
  i <> j ->
    select (store a i x) j =
    select a j

logic sorted : array -> int -> prop
axiom sorted_def: forall a,n.
  sorted a n <->
    forall i,j. 0 <= i <= j < n ->
      select a i <= select a j

logic swap : array -> array ->
  int -> int -> prop

axiom swap_def: forall a,b,i,j.
  swap a b i j <->
    select a i = select b j /\
    select a j = select b i /\
    forall k. k <> i /\ k <> j ->
      select a k = select b k

logic permut:
  array -> array -> int -> prop
axiom permut_refl: forall a,n.
  permut a a n
axiom permut_sym: forall a,b,n.
  permut a b n -> permut b a n
axiom permut_trans: forall a,b,c,n.
  permut a b n /\ permut b c n ->
    permut a c n
axiom permut_swap: forall a,b,i,j,n.
  0 <= i < n /\ 0 <= j < n /\
  swap a b i j -> permut a b n

```

Figure 1: Logical context for sorting

2 Logical Contexts

Our background logic is multi-sorted first-order logic with equality. Models for specifying programs can be defined by declaring types, constant, function and predicate symbols and axioms. Models may integrate predefined theories, a typical example being integer arithmetic.

2.1 Logical Signatures

Definition 1 A logical signature is composed of (1) a set *utype* of sort names introduced by the user ; (2) a set *sym* of constant, function and predicate symbols ; (3) a set *ref* of global reference names and (4) a set *exc* of exceptions names. The set of all data types is defined by the grammar

$$\text{type} ::= \text{Tuser } \text{utype} \mid \text{Tarrow } \text{type } \text{type} \mid \text{Tprop}$$

that is, the types are completed with built-in types for propositions and functions. We require every symbol, exception and reference to have an associated *type*. Our Coq implementation is as follows, where *sym*, *ref*, and *exc* are of type *type* $\rightarrow$ *Type*. The parameters of the latter are written as subscript in the following.

```

Variable utype : Type.
Inductive type : Type :=
| Tuser : utype -> type          (* user type name *)
| Tprop : type                  (* propositions *)
| Tarrow : type -> type -> type (* type of functional constants *)
.
Variable sym : type -> Type.
Variable ref : type -> Type.
Variable exc : type -> Type.

```

Example 2 Fig. 1 presents an appropriate model for specifying a program for sorting an array. An abstract type *array* is introduced to model arrays of integers indexed by integers. It is axiomatized with the well-known theory of arrays. We also define predicates (*sorted* *t* *i*) meaning that $t[0], \dots, t[i-1]$ is an increasing sequence, and (*permut* t_1 t_2) meaning that t_1 is a permutation of t_2 . The latter is axiomatized: it is an equivalence relation that contains all transpositions (*swap*) of two elements.

The logical signature of this example is thus given by $utype = \{array\}$ and $sym = \{select, store, sorted, swap, permut\}$ (ref and exc will come later). Each symbol is annotated by the appropriate type, e.g. $select : sym_{(Tarrow(Tuser\ array)(Tarrow\ Tint\ Tint))}$.

2.2 Dependently Typed *de Bruijn* Indices

A design choice in our formalization is to define terms and expressions such that they are well typed by construction. This simplifies the definition of the semantics and the weakest precondition calculus on such expressions, as we don't need to handle malformed constructions at those points. To begin we need to ensure that occurrences of variables actually correspond to bound variables in their current scopes and that they are used with the correct type. Here we use so-called dependently typed *de Bruijn* indices following the preliminary approach of Herms [15] as documented in [8].

Dependent indices are like regular *de Bruijn* indices, in that I_0 refers to the innermost bound variable, ($I_S\ I_0$) to the second innermost bound variable, etc. Additionally they carry information about their typing environment and about the type of the variable they represent. We use indices of type $idx_{A,E}$ to represent variables of type A under a typing environment E , that is the list of the types of the bound variables. The type of the innermost bound variable is stored at the first position in the typing environment, the type of the second innermost bound variable at the second position, etc. In Coq we can formalize this constraint using a parametrized annotated inductive type. The definition constrains the type of the first index to match the first element in the type list and recursively for the other elements.¹

```
Variable S : Type.
Variable T : S -> Type.
Inductive lidx A : list S -> Type :=
| HIO E : lidx A (A :: E)
| HIS B E : lidx A E -> lidx A (B :: E).
Inductive hlist : list S -> Type :=
| Hnil : hlist []
| Hcons A E : T A -> hlist E -> hlist (A :: E).
```

Then we can write the function `accslidx` which given an index and an `hlist` returns the element in the list pointed by the index. In Coq, thanks to the PROGRAM environment its definition is straightforward.

```
Program Fixpoint accslidx A E (i:lidx A E) (l:hlist E) : T A :=
  match l with
  | Hnil => !
  | Hcons _ _ h q =>
    match i with
    | HIO _ => h
    | HIS _ _ i1 => accslidx i1 q
    end
  end.
```

Dependent indices are thus placeholders within terms but they can also be used to reference elements within heterogeneous lists. In such a heterogeneous list each element may have a different type. The type $hlist_E$ of heterogeneous lists then depends on the list of types E of their elements. Thanks to the constraints on the type parameters, if an index $i : idx_{A,E}$ references an element within a heterogeneous list $l : hlist_E$, we are sure to find an element of type A at i -th position of l . This allows us to define the function $accsidx : idx_{A,E} \rightarrow hlist_E \rightarrow A$ which recursively accesses elements within a heterogeneous list.

We will use these heterogeneous lists to give semantics to our languages. Precisely, heterogeneous lists are the representation of evaluation environments which associate a value to each variable in the

¹This definition matches the Coq development where indexes are named `lidx` whereas in the article they are called idx , similarly `HIO` and `HIS` correspond to I_0 and I_S of the paper.

$t_{L,E,A} ::=$	$\text{Tconst } sym_A$	$p_{L,E} ::=$	$\text{Peq } t_{L,E,A} \ t_{L,E,A}$
	$\text{Tvar } idx_{A,E}$		$\text{Pand } p_{L,E} \ p_{L,E}$
	$\text{Tapp } t_{L,E,(\text{Tarrow } BA)} \ t_{L,E,B}$		$\text{Pimply } p_{L,E} \ p_{L,E}$
	$\text{Tlet } t_{L,E,B} \ t_{L,B::E,A}$		$\text{Pforall } p_{L,A::E}$
	$\text{Tderef } ref_A \quad (* \ ! \ r \ *)$		$\text{Plet } t_{L,E,A} \ p_{L,A::E}$
	$\text{Tat } label_L \ ref_A \quad (* \ r @ l \ *)$		Pfalse
			$\text{Pterm } t_{L,E,\text{Tprop}}$

Figure 2: Inductive definitions of terms and propositions

current typing environment. The function `accsidx` is then used in the semantics rule for variable access.

Example 3 The heterogeneous list $l = [5; true; succ]$ has type $hlist \ [\mathbb{Z}; bool; \mathbb{Z} \rightarrow \mathbb{Z}]$. De Bruijn indices $I_0 : idx_{\mathbb{Z}, [\mathbb{Z}; bool; \mathbb{Z} \rightarrow \mathbb{Z}]}$ and $I_S (I_S I_0) : idx_{(\mathbb{Z} \rightarrow \mathbb{Z}), [\mathbb{Z}; bool; \mathbb{Z} \rightarrow \mathbb{Z}]}$, are well-typed and can be used to access their values, e.g. $accsidx \ I_0 \ l = 5 : \mathbb{Z}$ and $accsidx \ (I_S \ I_0) \ l = true : bool$.

2.3 Terms and Propositions

Terms and propositions follow the usual classical first-order logic. For the need of programs, we add the declaration of local names using `let` binders, the access to a reference r (with concrete syntax `! r`) and the dereferencing of such a reference in a former state labeled by l (concrete syntax `r @ l`). Labels are represented by bounded integers and new labels can be declared at the expression level.

The concrete grammar for terms t and propositions p as used the example (Figures 1 and 5) is

$t ::=$	s	logic constant
	$(F \ t \ \dots \ t)$	symbol application
	$\text{let } v = t \text{ in } t$	local binding
	v	local name
	$! \ r$	dereferencing
	$r @ l$	dereferencing at label
$p ::=$	t	atomic proposition
	$\text{let } v = t \text{ in } p$	local binding
	$\neg p \mid p \wedge p \mid p \vee p \mid p \rightarrow p$	connectives
	$\forall v : \tau, p \mid \exists v : \tau, p$	quantifications

The formal syntax of terms and propositions is given in Fig. 2. Terms $t_{L,E,A}$ and propositions $p_{L,E}$ depend on the parameters E and L , denoting respectively the typing environment and the highest index of a valid label. Terms additionally depend on the parameter A , the type of the value they denote. Variables are represented by our dependent indices $idx_{A,E}$. The constructor `Tlet` expresses let-blocks at the term level. As usual with *de Bruijn* indices, no variable name is given and the body of the block is typed in a typing environment that is enriched by the type of the term to be remembered. The symbol application is formalized in a curried style. For the propositions we define only the ones needed within the **WP** calculus. The constructor `Pterm` allows to construct user-defined atomic propositions from terms. As `Tlet` at the term-level, `Plet` expresses let-blocks at the level of props and binds a new *de Bruijn* variable. Similarly `Pforall` binds a new *de Bruijn* variable but generalizing it instead of assigning a value to it. The `Pforall` and the `Plet` bind a new *de Bruijn* variable.

2.4 Logical Contexts, Semantics

The semantics of our generic language depends on an *interpretation* given to types and symbols. From such an interpretation, any term or proposition can be given a value, in a given *environment* for variables and given *state* for references.

Given a logical signature, an *interpretation* is a pair of a function *denutype* giving an interpretation of the user types, and a function *densym* giving an interpretation of the introduced function

$\llbracket \text{Tconst } s \rrbracket_{\Gamma, S} ::= \text{densym } s$	$\llbracket \text{Peq } t_1 \ t_2 \rrbracket_{\Gamma, S} ::= \llbracket t_1 \rrbracket_{\Gamma, S} = \llbracket t_2 \rrbracket_{\Gamma, S}$
$\llbracket \text{Tvar } v \rrbracket_{\Gamma, S} ::= \text{accsidx } \Gamma \ v$	$\llbracket \text{Pand } p_1 \ p_2 \rrbracket_{\Gamma, S} ::= \llbracket p_1 \rrbracket_{\Gamma, S} \wedge \llbracket p_2 \rrbracket_{\Gamma, S}$
$\llbracket \text{Tderef } r \rrbracket_{\Gamma, S} ::= \text{Here } S \ r$	$\llbracket \text{Pimply } p_1 \ p_2 \rrbracket_{\Gamma, S} ::= \llbracket p_1 \rrbracket_{\Gamma, S} \rightarrow \llbracket p_2 \rrbracket_{\Gamma, S}$
$\llbracket \text{Tapp } t_1 \ t_2 \rrbracket_{\Gamma, S} ::= (\llbracket t_1 \rrbracket_{\Gamma, S} \llbracket t_2 \rrbracket_{\Gamma, S})$	$\llbracket \text{Pforall } p \rrbracket_{\Gamma, S} ::= \forall b : B, \llbracket p \rrbracket_{b::\Gamma, S}$
$\llbracket \text{Tlet } t_1 \ t_2 \rrbracket_{\Gamma, S} ::= \llbracket t_2 \rrbracket_{\llbracket t_1 \rrbracket_{\Gamma, S}::\Gamma, S}$	$\llbracket \text{Plet } t \ p \rrbracket_{\Gamma, S} ::= \llbracket p \rrbracket_{\llbracket t \rrbracket_{\Gamma, S}::\Gamma, S}$
$\llbracket \text{Tat } l \ r \rrbracket_{\Gamma, S} ::= \text{At } S \ l \ r$	$\llbracket \text{Pfalse} \rrbracket_{\Gamma, S} ::= \perp$
	$\llbracket \text{Pterm } t \rrbracket_{\Gamma, S} ::= \llbracket t \rrbracket_{\Gamma, S}$

Figure 3: Denotational semantics of terms and propositions

and predicate symbols. Given *denutype* we define *dentype* to interpret all types. An *evaluation environment* Γ of type *env_E* is a heterogeneous list as described above. A *memory state* S of type *state_L* is a vector of size L of mappings from references *ref_A* to values of type *dentypeA*. The first element denotes the current state whereas the $(l + 1)$ -nth element denotes the state labeled by l . This is the reason for the L -parameter of terms and propositions. A term of type $t_{L,E}$ can be safely evaluated in a state of type *state_L*. As a special case, a *state₀* is only composed of the current state and $t_{0,E}$ cannot contain any labeled dereferenciation at all. The semantics of terms is defined by structural recursion (Fig. 3), where we use the syntactic sugar *Here* $S = S[0]$ and *At* $S \ l = S[l + 1]$ by analogy to the syntax. Note how the rules for *Tlet* and *Pforall* push the newly bound variable into Γ . Note also how correct typing is ensured by construction.

A *logical context* is a pair of a logical signature and a set of axioms over it. The programs that will be written in the next section will assume a given logical context. The goal is to prove them valid with respect to *any* interpretation which makes the axioms of that context valid: this will allow us to use various provers to discharge them.

3 The Core Programming Language

3.1 Informal Description

Our core language follows most of the design choices of the input language of *Why*. Indeed we reduce to an even more basic set of constructs, nevertheless remaining expressive enough to encode higher-level sequential algorithms. We follow an ML-style syntax; in particular there is no distinction between expressions and instructions. A program in this language is defined by a logical context and a finite set of function definitions, denoted f below, which can modify the global references of the context and can be mutually recursive.

Following again the *Why* design, our core language contains an exception mechanism, providing powerful control flow structures. As we will see these can be handled by weakest pre-condition calculus without major difficulty. A grammar for the concrete syntax of programs is given in Fig. 4. Loops are infinite ones, with a given invariant. The only way to exit them is by using exceptions. We use $e_1; e_2$ as a shortcut for $\text{let } v = e_1 \text{ in } e_2$ when the variable v is unused.

A definition of a function follows the structure

$$\text{let } f(x_1 : \tau_1, \dots, x_n : \tau_n) : \tau = \{ p \} e \{ q \}$$

where the predicates p and q are the pre- and the post-condition. The types are those declared in the logical context. In the post-condition, the reserved name *result* is locally bound and denotes the result of the function of type τ and label *Old* is bound to denote the pre-state. Note that exceptions are not supposed to escape function bodies. We could easily support such a feature by adding a family of post-conditions indexed by exceptions as in *Why* [12].

Example 4 In Fig. 5 is a program that sorts the global array t by the classical selection sort algorithm. Note the use of the exception *Break* to exit from the infinite loops. Note also the use of labels in annotations, allowing to specify assertions, loop invariants and post-conditions that link up various states of execution.

$e ::= t$	term
$\text{let } v = e \text{ in } e$	local binding
$f(t, \dots, t)$	function call
$\text{if } p \text{ then } e \text{ else } e$	conditional branching
$r := t$	assignment of a reference
$\text{label } l : e$	labeled expression
$\text{assert } \{p\} \text{ in } e$	local assertion
$\text{raise } ex(t)$	exception throwing
$\text{try } e \text{ catch } ex(v) e$	exception catching
$\text{loop } \{\text{invariant } p\} e$	infinite loop

Figure 4: Syntax of concrete expressions

```

ref t : array                                (* look for minimum value
                                           among t[i..n-1] *)
let swap(i:int, j:int) : unit =
  { true }
  let tmp = select !t i in
  t := store !t i (select !t j);
  t := store !t j tmp
  { swap !t t@Old i j }

ref mi, mv, i, j : int
exc Break : unit

let selection_sort(n:int) : unit =
  { n >= 1 }
  i := 0;
  try loop
    { invariant 0 <= !i < n /\
      sorted !t i /\
      permut !t t@Old n /\
      forall k1, k2.
        0 <= k1 < i <= k2 < n ->
          select !t k1 <= select !t k2 }
    if !i >= n-1
    then raise (Break ()) else ();
    mv := select !t !i; mi := !i;
    j := !i+1;
    try loop
      { invariant !i < !j /\
        !i <= !mi < n /\
        !mv = select !t !mi /\
        forall k. !i <= k < !j ->
          select !t k >= !mv }
      if !j >= n
      then raise (Break ()) else ();
      if select !t !j < !mv
      then (mi := !j ;
        mv := select !t !j)
      else ();
      j := !j + 1;
      catch Break(v) ();
    label Lab:
      swap(!i, !mi);
      assert { permut !t t@Lab n } in
      i := !i + 1;
      catch Break(v) ();
    { sorted !t n /\ permut !t t@Old n }

```

Figure 5: Selection sort in our core language

$e_{L,E,A} ::=$	Eterm $t_{L,E,A}$	pure term t
	Elet $e_{L,E,B} \ e_{L,B::E,A}$	let $v = e_1$ in e_2
	Eassign $ref_A \ t_{L,E,A}$	$r := t$
	Eassert $p_{L,E} \ e_{L,E,A}$	assert $\{ p \}$ in e
	Eraise $exc_A \ t_{L,E,A}$	raise $(ex \ t)$
	Eif $p_{L,E} \ e_{L,E,A} \ e_{L,E,B}$	if p then e_1 else e_2
	Eloop $p_{L,E} \ e_{L,E,B}$	loop { invariant p } e
	Etry $e_{L,E,A} \ exc_B \ e_{L,B::E,A}$	try e_1 catch $ex(v) \ e_2$
	Elab $e_{L+1,E,A}$	label l : e
	Ecall $f_{A,P} (t_{L,E,P_1}, \dots, t_{L,E,P_n})$	call to f

Figure 6: Inductive definition of expressions

3.2 Formal Syntax of Expressions

Like terms of the logic, expressions of programs are formalized by an inductive type $e_{L,E,A}$ depending on the parameters A , E and L , denoting respectively the evaluation type, the typing environment

$$\begin{array}{c}
\frac{}{\Gamma, S, \text{Eterm } t \Rightarrow S, \llbracket t \rrbracket_{\Gamma, S}} \quad \frac{\Gamma, S, e_1 \Rightarrow S', v \quad v :: \Gamma, S', e_2 \Rightarrow S'', o}{\Gamma, S, \text{Elet } e_1 e_2 \Rightarrow S'', o} \\
\\
\frac{}{\Gamma, S, \text{Eassign } r t \Rightarrow S[r/\llbracket t \rrbracket_{\Gamma, S}], \llbracket t \rrbracket_{\Gamma, S}} \quad \frac{\Gamma, S, e_1 \Rightarrow S', ex(v)}{\Gamma, S, \text{Elet } e_1 e_2 \Rightarrow S', ex(v)} \\
\\
\frac{\Gamma, S \uparrow, e \Rightarrow S', o}{\Gamma, S, \text{Elabel } e \Rightarrow S' \downarrow, o} \quad \frac{\llbracket p \rrbracket_{\Gamma, S} \quad \Gamma, S, e \Rightarrow S', o}{\Gamma, S, \text{Eassert } p e \Rightarrow S', o} \\
\\
\frac{\llbracket p \rrbracket_{\Gamma, S} \quad \Gamma, S, e_1 \Rightarrow S', o}{\Gamma, S, \text{Eif } p e_1 e_2 \Rightarrow S', o} \quad \frac{\neg \llbracket p \rrbracket_{\Gamma, S} \quad \Gamma, S, e_2 \Rightarrow S', o}{\Gamma, S, \text{Eif } p e_1 e_2 \Rightarrow S', o} \\
\\
\frac{\llbracket p \rrbracket_{\Gamma, S} \quad S, e \Rightarrow S', v \quad S', \text{Eloop } p e \Rightarrow S'', o}{S, \text{Eloop } p e \Rightarrow S'', o} \\
\\
\frac{\llbracket p \rrbracket_{\Gamma, S} \quad S, e \Rightarrow S', ex(v)}{S, \text{Eloop } p e \Rightarrow S', ex(v)} \quad \frac{}{\Gamma, S, \text{Eraise } ex t \Rightarrow S, ex(\llbracket t \rrbracket_{\Gamma, S})} \\
\\
\frac{S, e_1 \Rightarrow S', o \quad o \neq ex}{S, \text{Etry } e_1 ex e_2 \Rightarrow S', o} \quad \frac{S, e_1 \Rightarrow S', ex(v) \quad v :: \Gamma, S', e_2 \Rightarrow S'', o}{S, \text{Etry } e_1 ex e_2 \Rightarrow S'', o} \\
\\
\frac{\Gamma_f := [\llbracket t_1 \rrbracket_{\Gamma, S}, \dots, \llbracket t_n \rrbracket_{\Gamma, S}] \quad \llbracket \text{pre}_f \rrbracket_{\Gamma_f, S} \quad \Gamma_f, S, \text{body}_f \Rightarrow S', v \quad \llbracket \text{post}_f \rrbracket_{v :: \Gamma_f, S'}}{\Gamma, S, \text{Ecall } f(t_1, \dots, t_n) \Rightarrow S', v}
\end{array}$$

Figure 7: Operational semantics of terminating expressions

and the highest index of a valid label. Abstract syntax of expressions including comprehensive type annotations is given in Fig 6. Notice that variables v and label l are left implicit in the inductive definition thanks to *de Bruijn* representation. Additionally expressions depend on a parameter F meaning the list of *signatures* of the functions in the program the expression can appear in. A signature is a pair of the return type of the function and the list of the function's parameters. F appears within expressions in function calls where we use dependent indexes to refer to functions, $f_{A,P} := \text{id}_{X_{\langle A, P \rangle}, F}$. A function identifier is therefore an index pointing to an element with the signature $\langle A, P \rangle$ within a heterogeneous list of types F . This heterogeneous list $\text{hlist}_{\text{func } F, F}$ is precisely the representation of a program pr_F , where each element is a function $\text{func}_{F, \langle A, P \rangle}$.

A function $\text{func}_{F, \langle A, P \rangle}$ consists of a body $e_{F, 1, E, A}$, a pre-condition $p_{0, P}$ and a post-condition $p_{1, A :: P}$. In the pre-condition no labels may appear, hence its type has the parameter 0. In the post-condition we allow referring to the pre-state of a function call: in the syntax this corresponds to using the label *Old*. The post-condition may additionally refer to the result of the function, hence its type environment is enriched by A . Note that in the definition of programs the parameter F appears twice: once as parameter of hlist , to define the signatures of the functions in the program, and once as parameter of func to constrain expressions in function bodies to refer only to functions with a signature appearing in F . This way we ensure the well-formedness of the graph structure of programs.

3.3 Operational Semantics

The operational semantics is defined in big-step style following the approach of Leroy and Grall [19]. A first set of inference rules inductively defines the semantics of terminating expressions (Fig. 7) and a second set defines the semantics of non-terminating expressions, co-inductively (Fig. 8). Judgement $\Gamma, S, e \Rightarrow S', o$ expresses that in environment Γ and state S , the execution of expression e terminates, in a state S' with *outcome* o : either a normal value v or an exception $ex(v)$ where v is the value held by it. There are two rules for $\text{let } e_1 \text{ in } e_2$ depending on the outcome of e_1 . The rule for assignment uses the update operation $S[r/a]$ on states which replaces the topmost mapping for r in S . A labeled expression is evaluated in an enriched state $S \uparrow$ where the current state is copied on top of the vector. The resulting state $S' \downarrow$ is obtained by deleting the second position of the vector what corresponds to “forget” the previously copied current state. The rule for function calls requires the pre-condition to be valid in the starting state and, if the function terminates normally, the validity of the post-condition

$$\begin{array}{c}
\frac{\Gamma, S, e_1 \Rightarrow \infty}{\Gamma, S, \text{Elet } e_1 e_2 \Rightarrow \infty} \qquad \frac{\Gamma, S, e_1 \Rightarrow S', v \quad v :: \Gamma, S', e_2 \Rightarrow \infty}{\Gamma, S, \text{Elet } e_1 e_2 \Rightarrow \infty} \\
\\
\frac{\llbracket p \rrbracket_{\Gamma, S} \quad \Gamma, S, e_1 \Rightarrow \infty}{\Gamma, S, \text{Eif } p e_1 e_2 \Rightarrow \infty} \qquad \frac{\neg \llbracket p \rrbracket_{\Gamma, S} \quad \Gamma, S, e_2 \Rightarrow \infty}{\Gamma, S, \text{Eif } p e_1 e_2 \Rightarrow \infty} \\
\\
\frac{\Gamma, S \uparrow, e \Rightarrow \infty}{\Gamma, S, \text{Elabel } e \Rightarrow \infty} \quad \frac{\llbracket p \rrbracket_{\Gamma, S} \quad \Gamma, S, e \Rightarrow \infty}{\Gamma, S, \text{Eassert } p e \Rightarrow \infty} \quad \frac{\llbracket p \rrbracket_{\Gamma, S} \quad S, e \Rightarrow \infty}{S, \text{Eloop } p e \Rightarrow \infty} \\
\\
\frac{\llbracket p \rrbracket_{\Gamma, S} \quad S, e \Rightarrow S', v \quad S', \text{Eloop } p e \Rightarrow \infty}{S, \text{Eloop } p e \Rightarrow \infty} \\
\\
\frac{S, e_1 \Rightarrow \infty}{S, \text{try } e_1 \text{ catch } ex() e_2 \Rightarrow \infty} \qquad \frac{S, e_1 \Rightarrow S', ex(v) \quad v :: \Gamma, S', e_2 \Rightarrow \infty}{S, \text{try } e_1 ex e_2 \Rightarrow \infty} \\
\\
\frac{\Gamma_f := [\llbracket t_1 \rrbracket_{\Gamma, S}, \dots, \llbracket t_n \rrbracket_{\Gamma, S}] \quad \llbracket pre_f \rrbracket_{\Gamma_f, S} \quad \Gamma_f, S, \text{body}_f \Rightarrow \infty}{\Gamma, S, \text{Ecall } f(t_1, \dots, t_n) \Rightarrow \infty}
\end{array}$$

Figure 8: Operational semantics of non-terminating expressions

in the returning state to be valid too.

Judgement $\Gamma, S, e \Rightarrow \infty$ expresses that the execution of expression e does not terminate in environment Γ and state S . Its definition is straightforward: the execution of an expression diverges if the execution of a sub-expression diverges. The interesting cases are for the execution of a loop: starting from a given state S , it diverges either if its body diverges or if its body terminates on some state S' and the whole loop diverges starting from this new state. Of course, non-termination may be caused by infinite recursion of functions, too.

The main feature to notice is that execution blocks whenever an invalid assertion is met: the rules for assertions, loops and function calls are applicable only if the respective annotations are valid. Conversely, as everything is well-typed by construction, the only reason why an expression wouldn't execute is that one of its annotations isn't respected.

Definition 5 (Safe execution) *An expression e executes safely in environment Γ and state S , denoted $\Gamma, S, e \xRightarrow{\text{safe}}$, if either it diverges: $\Gamma, S, e \Rightarrow \infty$, or it terminates: $S', o, \Gamma, S, e \Rightarrow S', o$.*

A program respects its annotations if for each function f and any Γ, S such that $\llbracket pre_f \rrbracket_{\Gamma, S}$ have $\Gamma, S, \text{body}_f \xRightarrow{\text{safe}}$ and if $\Gamma, S, \text{body}_f \Rightarrow S', o$ then o is a normal outcome v such that $\llbracket post_f \rrbracket_{v::\Gamma, S'}$.

Our semantics is quite unusual, in particular it is not executable. Although, if annotations are removed then it becomes executable (indeed only if the propositional guards in if-then-else blocks are decidable) and coincides with a natural semantics. This approach makes obsolete a distinct set of rules for axiomatic semantics *à la* Hoare: the soundness of the verification condition generator will be stated using this definition of safe execution. Moreover this notion of safe execution is indeed stronger than the usual notion of partial correctness: a safe program that does not terminate will still satisfy its annotations forever.²

4 Weakest Precondition Calculus

4.1 Effect Inference

To carry out the weakest precondition calculus we need to know the *effect* of each expression, i.e. the references it may modify. The only expression to modify a reference is the assignment $r := t$, so we just need to collect all the assignments in the sub-expressions and this cannot be done by

²Total correctness is not considered in this paper; however it is clear that one could add annotations for termination checking: variants for loops and for recursive functions as in ACSL [4].

$$\begin{aligned}
\text{WP (Eterm } t) Q R \Gamma S &= Q S \llbracket t \rrbracket_{\Gamma, S} \\
\text{WP (Elet } e_1 e_2) Q R \Gamma S &= \text{WP } e_1 (\lambda S a, \text{WP } e_2 Q R (a :: \Gamma) S) R \Gamma S \\
\text{WP (Eassign } r t) Q R \Gamma S &= Q (S[r/\llbracket t \rrbracket_{\Gamma, S}]) \llbracket t \rrbracket_{\Gamma, S} \\
\text{WP (Eassert } p e) Q R \Gamma S &= \llbracket p \rrbracket_{\Gamma, S} \wedge \text{WP } e Q R \Gamma S \\
\text{WP (Eraise } ex t) Q R \Gamma S &= R S ex \llbracket t \rrbracket_{\Gamma, S} \\
\text{WP (Eif } p e_1 e_2) Q R \Gamma S &= (\llbracket p \rrbracket_{\Gamma, S} \rightarrow \text{WP } e_1 Q R \Gamma S) \\
&\quad \wedge (\neg \llbracket p \rrbracket_{\Gamma, S} \rightarrow \text{WP } e_2 Q R \Gamma S) \\
\text{WP (Eloop } p e) Q R \Gamma S &= \llbracket p \rrbracket_{\Gamma, S} \wedge \forall S', S \xrightarrow{\text{writes } e} S' \rightarrow \llbracket p \rrbracket_{\Gamma, S'} \rightarrow \\
&\quad \text{WP } e (\lambda S'' v, \llbracket p \rrbracket_{\Gamma, S''}) R \Gamma S' \\
\text{WP (Etry } e_1 ex e_2) Q R \Gamma S &= \text{WP } e_1 Q (\lambda S' ex' a, \text{if } ex = ex' \\
&\quad \text{then WP } e_2 Q R (a :: \Gamma) S' \text{ else } R ex' a) \Gamma S \\
\text{WP (Elabel } e) Q R \Gamma S &= \text{WP } e Q R \Gamma S^\uparrow \\
\text{WP (Ecall } f (t_1, \dots, t_n)) Q R \Gamma S &= \llbracket pre_f \rrbracket_{\Gamma_{args}, S} \wedge \forall S' a, S \xrightarrow{\text{writes } f} S' \rightarrow \\
&\quad \llbracket post_f \rrbracket_{(a :: \Gamma_{args}), (S', S)} \rightarrow Q S' a \\
&\quad \text{where } \Gamma_{args} := [\llbracket t_1 \rrbracket_{\Gamma, S}, \dots, \llbracket t_n \rrbracket_{\Gamma, S}]
\end{aligned}$$

Figure 9: Recursive definition of the WP-calculus

structural recursion because of the function calls. Calling a function means possibly modifying all the references appearing in assignments inside the function's body, so we need to know them. Since the functions can be mutually recursive, we compute their effects by iterating a function *writes* collecting additional effects, until it reaches a fixpoint. Termination of this algorithm has been proven in Coq.

Definition 6 *The effects ϵ of a program associate a finite set of references to each function of the program.*

*Given the current effects ϵ , the function *writes* of type $\text{effects} \rightarrow e_{L, E, A} \rightarrow \text{rset}$ recursively collects the references modified in the given expression assuming ϵ_f as the references modified by the function f . If $\epsilon_f = \text{writes } \epsilon \text{ body}_f$ for every f , then ϵ is correct for the program.*

*The function *infer* of type $\text{pg}_F \rightarrow \text{effects}$ computes this fixpoint. Starting from the empty effects ϵ_0 , it computes $\epsilon_{n+1} := \text{map } (\lambda f. \text{writes } \epsilon_n \text{ body}_f) \text{ pg}$ until $\epsilon_{n+1} = \epsilon_n$.*

In the following we will simply write ϵ for $(\text{infer } \text{pg})$ and *writes* for $(\text{writes } (\text{infer } \text{pg}))$, because the effects can be computed once and for all for a given program.

Lemma 7 (Soundness of effects inference) *For all program pg and all function index $f : \text{idX}_{\langle A, P \rangle, F}$, $\epsilon_f = \text{writes } \text{body}_f$.*

Definition 8 *The relation *assigns* on states, denoted $S \xrightarrow{s} S'$, is true whenever S and S' differ only in the references appearing in the set s :*

$$S \xrightarrow{s} S' := \forall r : \text{ref}_A, r \notin s \rightarrow (\text{Here } S' r = \text{Here } S r \wedge \forall l, \text{At } l S' r = \text{At } l S r).$$

Lemma 9 *If $\Gamma, S, e \Rightarrow S'$, then $S \xrightarrow{\text{writes } e} S'$.*

4.2 Definition of the WP-calculus

We calculate the weakest pre-condition of an expression given a post-condition by structural recursion over expressions (Fig. 9). We admit several post-conditions,

$$\text{Normal}_{L, A} : \text{state}_L \rightarrow \text{dentype}_A \rightarrow \text{Prop}$$

for regular execution and

$$Exceptional_L : state_L \rightarrow \forall B, exn_B \rightarrow dentype_B \rightarrow \text{Prop}$$

for exceptional behavior. So our calculus has the type

$$\text{WP} : e_{L,E,A} \rightarrow Normal_{L,A} \rightarrow Exceptional_L \rightarrow env_E \rightarrow state_L \rightarrow \text{Prop}.$$

In the case of a loop, the pre-condition is calculated using the loop invariant and in the case of a function call we use the pre- and post-condition of that function. In both cases, as it is classical in WP calculi, we need to quantify over all states that may be reached by normal execution starting from the given state S : these are the states S' which differ from S only for the references that are modified in the loop or the function's body. The set of modified references is computed using our effect inference explained above.

Definition 10 *The verification conditions, respectively for one function and for a whole program, are*

$$\begin{aligned} VC(f) &:= \llbracket pre_f \rrbracket_{\Gamma, S} \rightarrow \text{WP } body_f (\lambda S' v, \llbracket post_f \rrbracket_{v::\Gamma, S'}) (\lambda S' ex v, \text{False}) \Gamma S \\ VCGEN &:= \forall f : id_{x_{\langle A, P \rangle, F}} \Gamma S, VC(f) \end{aligned}$$

The False as exceptional post-conditions requires that no function body exits with an exception.

4.3 Soundness Results

A preliminary property to establish is that after a terminating execution, post-conditions are respected if the weakest pre-condition is valid. It is proved by induction over the derivation of $\Gamma, S, e \Rightarrow S', o$.

Lemma 11 *For all environment Γ , expression $e_{L,E,A}$, initial and final states S, S' , outcome o and post-conditions $Q_{L,A}, R_L$, if $(\text{WP } e Q R \Gamma S)$ and $\Gamma, S, e \Rightarrow S', o$ then (1) if o is a normal outcome v then $(Q S' v)$, and (2) if o is an exceptional outcome $ex(v)$ then $(R S' ex v)$*

We now state that if the VCs hold for all functions then any expression having a valid WP executes safely. It is proved by co-induction, using the axiom of excluded middle to distinguish whether the execution of an expression does or does not terminate, following the guidelines of Leroy and Grall [19]. Notice that it is enough to prove the verification conditions for each function separately, even if functions can be mutually recursive, there is no circular reasoning.

Lemma 12 (safety of expressions) *If VCGEN holds then for any Γ, S, e, Q, R , if $(\text{WP } e Q R \Gamma S)$ then $\Gamma, S, e \xRightarrow{\text{safe}}$.*

The important corollary below states that if the VCs hold for all functions then their bodies all execute safely. By definition of the semantics, this implies that all assertions, invariants and pre- and post-conditions in a given program are verified if the verification conditions are valid.

Theorem 13 (soundness of VCGEN) *If VCGEN holds then the program respects its annotations, as defined in Def. 5*

5 Extraction of a Certified Verification Tool

The obtained Coq function for generating verification conditions is not *extractable*: given a program pg we obtain a Coq term $(VCGEN pg)$ of Coq type Prop which must be proved valid to show the correctness of the program. The process thus remains based on Coq for making the proofs. In this section we show how to extract the calculus into a separate tool so that proofs can be performed with other provers, e.g. SMT solvers.

5.1 Concrete WP computation

To achieve this we need the WP calculus to produce a formula in the abstract syntax of Fig. 2 instead of a Coq Prop. We define another function

$$\mathbf{wp} : e_{L,E,A} \rightarrow p_{L,A::E} \rightarrow (exn_B \rightarrow p_{L,B::E}) \rightarrow p_{L,E}$$

which, given an expression e , a normal post-condition Q and a family of exceptional post-conditions R , returns a weakest pre-condition. It is defined recursively on e similarly to **WP** in Fig. 9, but this time Q , R and the result are syntactic propositions which are concretely transformed. The concrete **wp** calculus is similar to the **WP**, the notable difference is that quantification over states is now replaced but several quantifications over modified references.

```

Program Fixpoint wp L E A (e:expr L E A): forall (Q:prop L (A::E))
(R: forall Aex, exn Aex -> prop L (Aex::E)), prop L E :=
  match e return _ with
  | Eterm t => fun Q R, Plet t Q
  | Elet A1 e1 e2 => fun Q R,
    wp e1 (wp e2 (lift_prop (E1:=[A]) Q)
      (fun A ex, lift_prop (E1:=[A]) (R A ex))) R
  | Eassign r t => fun Q R, Plet (t) (subst_prop Q r (Tvar HI0))
  | Eraise Aex ex t => fun Q R, Plet t (R Aex ex)
  | Eassert P e => fun Q R, Pand P (wp e Q R)
  | Eif t e1 e2 => fun Q R, Pand (Pimply p (wp e1 Q R))
    (Pimply (Pimply p Pfalse) (wp e2 Q R))
  | Eloop Inv e1 => fun Q R, Pand Inv
    (abstr (Pimply Inv
      (wp e1 (lift_prop (E1:=[] ) Inv) R)) (writes_ e1))
  | Etry Aex e1 ex e2 => fun Q R,
    wp e1 Q
    (fun Aex' ex' => if ex' `== ex then
      cast (T:=prop L) ((wp e2 ((lift_prop (E1:=[A]) Q))
        (fun Aex'' ex'', (lift_prop (A0:=Aex) (E1:=[Aex''])
          (E2:=E) (R Aex'' ex'')))))
      else R Aex' ex')
  | Elab e => fun Q R, dnlab_prop (wp e (uplab_prop Q)
    (fun A ex, uplab_prop (R A ex)))
  | Ecall P fi ps => fun Q R,
    let f := accsfunc fi in
    Pand (uplabn_prop (substps_prop ps (E1:=[]) (f.(pre))))
    (dnlab_prop (abstr
      (Pforall (Pimply
        (uplabn_prop (substps_prop
          (Indexes.map (T2:=term 1 E)
            (fun A (t:term 0 E A), uplabx t) ps)
          (E1:=[A]) (f.(post)))))
        (uplab_prop Q)))
      (accslist fi F_effects))))
  end.

```

Lemma 14 If $\llbracket \mathbf{wp} \, e \, Q \, R \rrbracket_{\Gamma, S}$ then

$$\mathbf{WP} \, e \, (\lambda S \, v, \llbracket Q \rrbracket_{v::\Gamma, S}) \, (\lambda S \, ex \, v, \llbracket R \, ex \rrbracket_{v::\Gamma, S}) \, \Gamma \, S$$

From **wp** we now define a concrete verification-condition generator **vcgen**.

Definition 15 The concrete VCs of a program pg is the list $(vcgen\ pg)$ of concrete formulas $(Abstr(Pimply\ pre_f\ (\mathbf{wp}\ body_f\ post_f\ Pfalse)))$ for each function f of pg . $Abstr$ is a generalization function: it prefixes any formula $p_{L,E}$ by as many $Pforall$ as elements of E to produce a $p_{L,\square}$ formula.

Theorem 16 If for all p in the list $(vcgen\ pg)$ and for all state S , $\llbracket p \rrbracket_{\square,S}$ then $(VCGEN\ pg)$.

That is, the hypothesis of Theorem 13 is valid if we prove the formulas generated by $vcgen$ valid in any state.

5.2 Producing Concrete Syntax with Explicit Binders

Still, formulas of $vcgen$ are represented by a de Bruijn-style abstract syntax. To print out such formulas we need to transform them into concrete syntax with identifiers for variables by generating new names for all the binders. This could be done on the fly in an unproven pretty-printer. Though, being a non trivial transformation it is better to do it in a certified way directly after the generation.

We therefore formalize a back-end syntax, along with its semantics for well-typed terms and propositions. It is similar to Fig. 2 where we replace $Tconst$, $Tvar$ and $Tderef$ by a new constructor $Tvar$ with an identifier as argument, and $Tlet$ and $Pforall$ binders are also given an explicit identifier. Here are the inductive definitions for the back-end syntax, without de-bruijn index but with named binders.

		p	$::=$	$Peq\ t\ t$			
			$ $	$Pand\ p\ p$			
t	$::=$	$Tvarid$	$ $	$Pimply\ p\ p$	$task$	$::=$	$\{$
	$ $	$Tapp\ t\ t$	$ $	$Pforall\ id:ty.\ p$		$tenv :$	$dict\ id\ type$
	$ $	$Tlet\ id : ty = t\ in\ t$	$ $	$Plet\ id : ty = t\ in\ p$		$hypos :$	$list\ p$
			$ $	$Pfalse$		$goals :$	$list\ p\ }$
			$ $	$Pterm\ t$			

We define a compilation from *de Bruijn*-style terms and propositions to the back-end syntax and prove preservation of semantics.

Finally, we define a *proof task* as a triple (d, h, g) where d is a finite map from identifiers to their type, h is a set of hypotheses and g is a list of goals. Such a task is said valid if the goals are logical consequences of the hypotheses, whatever the interpretation of symbols in d . The complete process of VC generation is to produce, from a logical context C and a program pg , the proof task $T(C, pg) = (d, h, g)$ where d are the declarations of C that appear in pg , h the compilation of axioms of C , and g is the compilation of $vcgen(pg)$.

Theorem 17 (Main soundness theorem) For all logical context C and program p , if the proof task $T(C, p)$ is valid then for any interpretation of the context in which the axioms are valid, p executes safely.

Notice that this statement is independent of the underlying proof assistant **Coq**: the validity of logical formulas in the proof task can be established by any theorem prover. The only hypothesis is that the backend theorem prover in use must agree with our definition of the interpretation of logical contexts. But this is just the classical first-order logic with equality, with standard predefined theories like integer arithmetic. All the off-the-shelf theorem provers, e.g SMT solvers, agree on that.

5.3 Extraction and Experimentation

For experimentation purposes we also defined a compilation in the opposite direction, i.e. from programs in front-end syntax to the corresponding program in *de Bruijn* syntax, provided that the former is well typed. We then use the extraction mechanism of **Coq** to extract an Ocaml function that, given an AST of our front-end syntax representing a program, produces a list of ASTs representing the proof task. We finally combine this with the Why3 parser for input programs and a hand-written pretty-printer that produces Why3 syntax [6], allowing us to call automated provers on the proof task.

We made experiments to validate this process. On our selection sort example, the two VCs for functions `swap` and `selection_sort` are generated in a fraction of a second by the standalone VC generator. These are sent to the Why3 tool, and they are proved automatically, again in a fraction of a second, by a combination of SMT solvers (i.e. after splitting these formulas, which are conjunctions, into parts [6]). For details see the Coq development at the URL given in the introduction.

6 Conclusions, Related Works and Perspectives

We formalized a core language for deductive verification of imperative programs. Its operational semantics is defined co-inductively to support possibly non-terminating functions. The annotations are taken into account in the semantics so that validity of a program with respect to its annotations is by definition the progress of its execution. We used an original formalization of binders so that only well-typed programs can be considered, allowing us to simplify the rest of the formalization. Weakest precondition calculus is defined by structural recursion, even in presence of mutually recursive functions, assuming the given function contracts. Even if there is an apparent cyclic reasoning, this approach is shown sound by a co-inductive proof. By additionally formalizing an abstract syntax for terms and formulas, and relating their semantics with respect to the Coq propositions, we defined a concrete variant of the WP calculus which can be extracted to OCaml code, thus obtaining a trustable and executable VC generator close to Why or Boogie.

As explained in the introduction, two kinds of approaches for deductive verification exist depending on the use of a deep embedding of the programming language or not. The approaches without deep embedding typically allows the user to discharge proof obligations using automatic provers, but are not certified correct. Our work fills this gap. Among deep-embedding-based approaches, the SunRise system of Homeier et al. [16, 17] is probably the first certified program verifier, and uses a deep embedding in the HOL proof environment. They formalize a core language and its operational semantics, and prove correct a set of Hoare-style deduction rules. Programs are thus specified using HOL formulas and proved within the HOL environment. Later Schirmer [25] formalized a core language in Isabelle/HOL, and Norrish formalized the C programming language [23], with similar approaches. More recently, similar deep-embedding-based approaches were proposed using Coq like in the Ynot system [22, 9], which can deal with “pointer” programs via separation logic, and also supports higher-functions.

A major difference between the former approaches and ours is that we use a deep embedding not only for programs but also for propositions and thus for specifications. This allows us to extract a standalone executable, and consequently to discharge VCs using external provers like SMT solvers. Our approach is a basis to formalize specification languages like JML and ACSL defined on top of mainstream programming language, which allows a user to specify and prove Java or C programs without relying on the logic language of a specific proof assistant.

Another difference is that we do not consider any Hoare-style rules but formalize a Dijkstra-style VC generator instead. This way to proceed is motivated by the choice of defining the meaning of “a program satisfies its annotations” by safety of its execution.

There are also some technical novelties in our approach with respect to the systems mentioned above. Our core language supports exceptions, which is useful for handling constructs of front-ends like `break` and `continue`, or Java exceptions. Specifications can also use labels to refer to former states of executions, with constructs like `\old` and `\at` constructs of JML and ACSL. This provides a handy alternative to the so-called *auxiliary* or *ghost* variables used in deep-embedding-based systems above. Indeed in the context of VC generation instead of Hoare-style rules, the semantics of such variables is tricky, e.g. when calling a procedure, the ghost variables should be existentially quantified, which results in VCs difficult to solve by automated provers. We believe that the use of labels is thus better.

Our main future work is to certify the remaining part of a complete chain from ACSL-annotated C programs to proof obligations. A first step is the formalization of a front-end like Frama-C/Jessie which compiles annotated C to intermediate Why code. We plan to reuse the C semantics defined in CompCert [18] and incorporate ACSL annotations into it. The main issue in this compilation process is the representation of the C memory heap by Why global references using a memory heap

modeling. In particular, first-order modeling of the heap, mainly designed to help automatic provers, raised consistency problems in the past [26]. In our approach where the axioms of the logical context are realized in Coq, the consistency is guaranteed. Finally, another part of the certification of the tool chain is the certification of back-end automatic provers, for which good progress was obtained recently, see e.g. [20].

Acknowledgments

We would like to thanks Jean-Christophe Filliâtre and Julien Signoles for their remarks on a preliminary version of this paper.

References

- [1] J.-R. Abrial. *The B-Book, assigning programs to meaning*. Cambridge University Press, 1996.
- [2] M. Barnett, R. DeLine, B. Jacobs, B.-Y. E. Chang, and K. R. M. Leino. Boogie: A Modular Reusable Verifier for Object-Oriented Programs. In *4th FMCO*, volume 4111 of *LNCS*, pages 364–387, 2005.
- [3] M. Barnett, K. R. M. Leino, and W. Schulte. The Spec# Programming System: An Overview. In *CASSIS'04*, volume 3362 of *LNCS*, pages 49–69. Springer, 2004.
- [4] P. Baudin, J.-C. Filliâtre, C. Marché, B. Monate, Y. Moy, and V. Prevosto. *ACSL: ANSI/ISO C Specification Language, version 1.4*, 2009. <http://frama-c.cea.fr/acsl.html>.
- [5] Y. Bertot and P. Castéran. *Interactive Theorem Proving and Program Development*. Springer-Verlag, 2004.
- [6] F. Bobot, J.-C. Filliâtre, C. Marché, and A. Paskevich. Why3: Shepherd your herd of provers. In *Boogie 2011: First International Workshop on Intermediate Verification Languages*, Wrocław, Poland, August 2011.
- [7] L. Burdy, Y. Cheon, D. R. Cok, M. D. Ernst, J. R. Kiniry, G. T. Leavens, K. R. M. Leino, and E. Poll. An overview of JML tools and applications. *International Journal on Software Tools for Technology Transfer*, 7(3):212–232, June 2005.
- [8] A. Chlipala. *Certified Programming with Dependent Types*. MIT Press, 2011. <http://adam.chlipala.net/cpdt/>.
- [9] A. J. Chlipala, J. G. Malecha, G. Morrisett, A. Shinnar, and R. Wisnesky. Effective interactive proofs for higher-order imperative programs. In *ICFP*, pages 79–90. ACM, 2009.
- [10] D. R. Cok and J. Kiniry. ESC/Java2: Uniting ESC/Java and JML. In *CASSIS'04*, volume 3362 of *LNCS*, pages 108–128. Springer, 2004.
- [11] M. Dahlweid, M. Moskal, T. Santen, S. Tobies, and W. Schulte. VCC: Contract-based modular verification of concurrent C. In *ICSE'09*, pages 429–430. IEEE Comp. Soc. Press, 2009.
- [12] J.-C. Filliâtre. Verification of non-functional programs using interpretations in type theory. *Journal of Functional Programming*, 13(4):709–745, July 2003.
- [13] J.-C. Filliâtre and C. Marché. Multi-prover verification of C programs. In *Sixth ICFEM*, pages 15–29. Springer, 2004.
- [14] J.-C. Filliâtre and C. Marché. The Why/Krakatoa/Caduceus platform for deductive program verification. In *CAV*, volume 4590 of *LNCS*, pages 173–177. Springer, July 2007.
- [15] P. Herms. Certification of a chain for deductive program verification. In Y. Bertot, editor, *2nd Coq Workshop, satellite of ITP'10*, 2010.

- [16] P. V. Homeier and D. F. Martin. A mechanically verified verification condition generator. *The Computer Journal*, 38(2):131–141, July 1995.
- [17] P. V. Homeier and D. F. Martin. Mechanical verification of mutually recursive procedures. In *13th CADE*, volume 1104 of *LNAI*, pages 201–215. Springer, 1996.
- [18] X. Leroy. A formally verified compiler back-end. *Journal of Automated Reasoning*, 43(4):363–446, 2009.
- [19] X. Leroy and H. Grall. Coinductive big-step operational semantics. *Inf. Comput.*, 207:284–304, February 2009.
- [20] S. Lescuyer. *Formalisation et développement d’une tactique réflexive pour la démonstration automatique en Coq*. Thèse de doctorat, Université Paris-Sud, 2011.
- [21] C. Marché, C. Paulin-Mohring, and X. Urbain. The KRAKATOA tool for certification of JAVA/JAVACARD programs annotated in JML. *Journal of Logic and Algebraic Programming*, 58(1–2):89–106, 2004. <http://krakatoa.lri.fr>.
- [22] A. Nanevski, G. Morrisett, A. Shinnar, P. Govereau, and L. Birkedal. Ynot: Reasoning with the awkward squad. In *ICFP’08*, 2008.
- [23] M. Norrish. *C Formalised in HOL*. PhD thesis, University of Cambridge, Nov. 1998.
- [24] F. Randimbivololona, J. Souyris, P. Baudin, A. Pacalet, J. Raguideau, and D. Schoen. Applying formal proof techniques to avionics software: A pragmatic approach. In *Formal Methods*, volume 1709 of *LNCS*, pages 1798–1815. Springer, 1999.
- [25] N. Schirmer. *Verification of Sequential Imperative Programs in Isabelle/HOL*. PhD thesis, Technische Universität München, 2006.
- [26] M. Wagner and T. Bormer. Testing a verifying compiler. In *1st FoVeOOS*, Karlsruhe Reports in Informatics, 2010. <http://digbib.ubka.uni-karlsruhe.de/volltexte/1000019083>.



**RESEARCH CENTRE
SACLAY – ÎLE-DE-FRANCE**

Parc Orsay Université
4 rue Jacques Monod
91893 Orsay Cedex

Publisher
Inria
Domaine de Voluceau - Rocquencourt
BP 105 - 78153 Le Chesnay Cedex
inria.fr

ISSN 0249-6399